

AMRITA SCHOOL OF ENGINEERING, BENGALURU

DEPARTMENT OF ELECTRONICS & COMMUNICATION
ENGINEERING

B. TECH. II SEMESTER(ECE) 23ECE184
INTRODUCTION TO INTERNET OF THING
LAB
SYSTEM DESIGN REPORT

Submitted by

S.V.CHIRUDEEP REDDY
DEV REHANTH
U.GANESH

BL.EN.U4ECE24164
BL.EN.U4ECE24155
BL.EN.U4ECE24157

BATCH NUMBER:0

Faculty Examiner Signature with Date

TITLE: Smart Dust Bin

OBJECTIVE: To create an IoT smart dustbin that ensures hygienic, touchless waste disposal and real-time monitoring of fill levels.

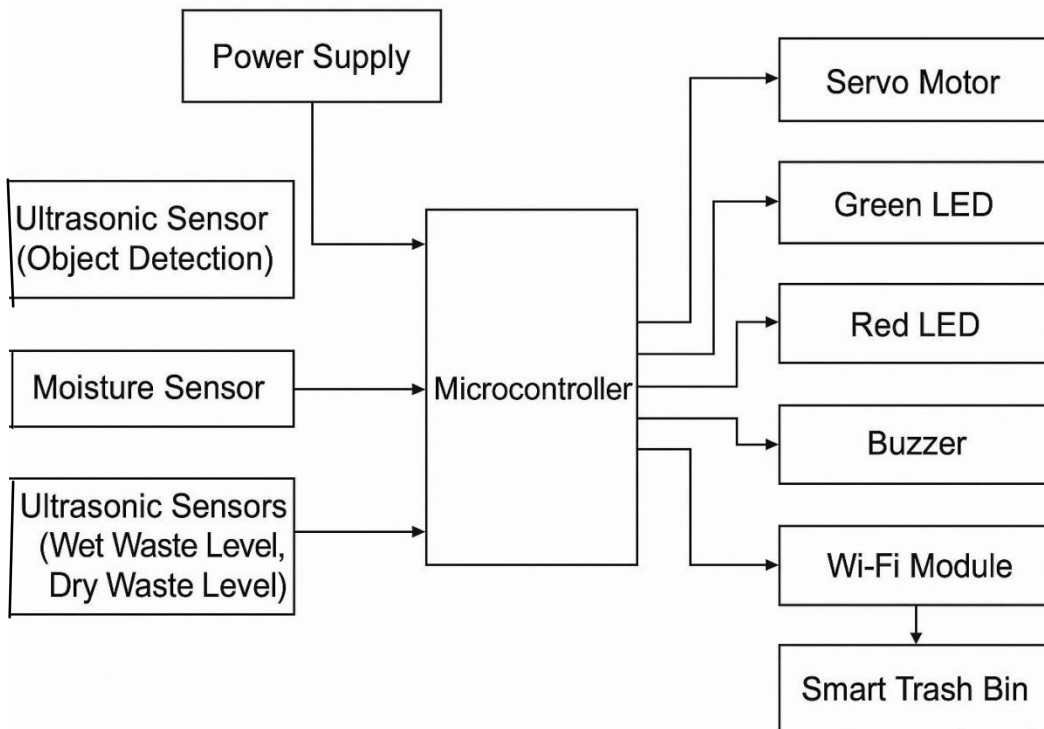
COMPONENTS REQUIRED ALONG WITH THEIR SPECIFICATIONS:

Sl. No	Name of the Component	Specification	Quantity
1.	Wi-fi Module	ESP-32	1
2.	Ultrasonic sensor	HC-SR04	3
3.	Servo Motor	SG-90	3
4.	Soil Moisture sensor	HW-038	1
5.	Jumper Wires	-	-
6	LEDS	RED, GREEN	2

MOTIVATION/SOCIATAL/INDUSTRIAL IMPACT:

The driving force to create a Smart Dustbin through IoT is due to the increasing problems in municipal waste management, including overflowing dustbins, inefficient manual checking, and the requirement for smart city infrastructure. Through sensors and connectivity, the system can track bin levels in real time, thereby facilitating timely waste collection and maintaining cleaner environments. The social benefit comprises enhanced hygiene, lower pollution, and enhanced public health, as well as greater awareness of proper waste disposal. Industrially, smart dustbins assist in streamlining waste collection routes, lowering operational expenses, and offering valuable insights for planning. They also aid in the bigger picture of creating smart cities and create opportunities for innovation in IoT and environmental tech industries.

BLOCK DIAGRAM/SYSTEM DESIGN:



CODE (ARDUNIO AND MIT APP CODE):

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

// WiFi credentials
const char* ssid = "chiru";
const char* password = "12345678";
// --- Pin definitions ---
// Ultrasonic Sensors
#define WET_TRIG 12
#define WET_ECHO 13
#define DRY_TRIG 14
#define DRY_ECHO 27
#define OBJ_TRIG 25
#define OBJ_ECHO 26

// Other Sensors/Actuators
#define MOISTURE_PIN 34 // Analog input for moisture sensor
#define SORTING_SERVO_PIN 15 // Original servo for directing waste
#define WET_LID_SERVO_PIN 18 // NEW servo for Wet Lid
#define DRY_LID_SERVO_PIN 19 // NEW servo for Dry Lid
#define GREEN_LED 16
#define RED_LED 17
#define BUZZER 4

// Sensor parameters
const float BIN_HEIGHT = 17.0; // Bin height in cm
const float OBJ_THRESHOLD = 6.0; // Object detection threshold in cm
```

```

const float LEVEL_THRESHOLD = 4.0; //
Waste level full threshold in cm

const float MOISTURE_THRESHOLD = 10.0; //
Moisture percentage threshold

const int DETECTION_DELAY = 3000; // 3-
second delay after detection

const int SENSOR_READ_DELAY = 100; //
Delay between sensor readings in ms

const int SENSOR_SAMPLES = 3; // Number of
samples for averaging

// Servo parameters

const int SORTING_SERVO_HOLD_TIME =
2000; // Sorting servo hold time in ms

const int LID_CLOSED_ANGLE = 0; // Angle
for closed lids

const int LID_OPEN_ANGLE = 90; // Angle for
open lids

// --- Global variables ---

float wetLevel = 0.0;

float dryLevel = 0.0;

String lastWasteType = "None";

String prevWasteType = "None";

struct Detection {
    String type;

    String timestamp;
};

Detection detectionLog[50]; // Store up to 50
detections

int detectionCount = 0;

unsigned long lastDetectionTime = 0;

unsigned long sortingServoHoldStart = 0;

bool sortingServoHolding = false;

bool isWetLidOpen = false; // NEW: Track wet
lid state

bool isDryLidOpen = false; // NEW: Track dry
lid state

// --- Servo Objects ---

```

```

Servo sortingServo; // Original servo

Servo wetLidServo; // NEW: Wet lid servo

Servo dryLidServo; // NEW: Dry lid servo

// --- Web Server ---

WebServer server(80);

// --- Function Declarations ---

float getDistance(int trigPin, int echoPin);

float calculateLevel(float distance);

String getTimestamp();

String getHTML();

void handleRoot();

void handleData();

void handleStatus(); // Kept for potential
compatibility, though maybe redundant now

void handleHistory();

void handlePlainData(); // Kept for potential
compatibility

void handleLidControl(); // NEW: Handler for lid
buttons

// --- Sensor Functions ---

// Measure distance using ultrasonic sensor with
averaging

float getDistance(int trigPin, int echoPin) {
    float totalDistance = 0.0;

    for (int i = 0; i < SENSOR_SAMPLES; i++) {
        digitalWrite(trigPin, LOW);

        delayMicroseconds(2); // Ensure proper timing

        digitalWrite(trigPin, HIGH);

        delayMicroseconds(10);

        digitalWrite(trigPin, LOW);

        long duration = pulseIn(echoPin, HIGH,
30000); // Timeout after 30ms

        if (duration > 0) { // Only add valid readings

            totalDistance += (duration * 0.034) / 2; //

```

```

Distance in cm
    } else {

    }

    delay(10); // Small delay between samples
}

// Basic check: if totalDistance is very small
after loop, it might mean all readings failed.

if (totalDistance < 0.1 &&
    SENSOR_SAMPLES > 0) {

    // Consider returning a default/error value
    like BIN_HEIGHT or -1.0

    return BIN_HEIGHT; // Assume no object /
    max distance if readings failed

}

return totalDistance / SENSOR_SAMPLES; //
Average distance
}

// Calculate waste level as percentage
float calculateLevel(float distance) {

    if (distance < 0) return 0.0; // Handle potential
    error from getDistance

    if (distance <= LEVEL_THRESHOLD) return
    100.0; // Bin full at 4 cm or less

    if (distance > BIN_HEIGHT) distance =
    BIN_HEIGHT; // Cap distance at bin height

    return ((BIN_HEIGHT - distance) /
    BIN_HEIGHT) * 100.0; // Percentage

}

// Get timestamp (approximate, based on uptime)
String getTimestamp() {

    unsigned long totalSeconds = millis() / 1000;

    unsigned long seconds = totalSeconds % 60;

    unsigned long totalMinutes = totalSeconds / 60;

    unsigned long minutes = totalMinutes % 60;

```

```

    unsigned long hours = totalMinutes / 60;

    char buffer[16];

    sprintf(buffer, "%02lu:%02lu:%02lu", hours,
    minutes, seconds);

    return String(buffer);
}

// --- Web Interface ---

// Generate the HTML for the dashboard
String getHTML() {

    String html = "<!DOCTYPE
    html><html><head>";

    html += "<meta name='viewport'
    content='width=device-width, initial-scale=1'>";

    html += "<title>Smart Dustbin
    Dashboard</title>";

    html += "<style>";

    html += "body { font-family: Arial, sans-serif;
    margin: 0; padding: 20px; background:
    #f0f0f0; }";

    html += "h1 { color: #00796b; text-align: center;
    }";

    html += ".container { max-width: 800px;
    margin: 0 auto; }";

    html += ".card { background: white; padding:
    20px; margin: 10px 0; border-radius: 8px; box-
    shadow: 0 2px 4px rgba(0,0,0,0.1); }";

    html += ".progress { height: 20px; background:
    #e0e0e0; border-radius: 4px; overflow: hidden;
    margin-bottom: 10px; }"; // Added margin

    html += ".progress-bar { height: 100%;
    transition: width 0.5s ease; }";

    html += ".wet .progress-bar { background:
    #26a69a; }"; // Teal for wet

    html += ".dry .progress-bar { background:
    #ff9800; }"; // Orange for dry

    html += ".status, .lid-status { padding: 5px 10px;
    border-radius: 4px; display: inline-block; margin-
    left: 5px; }";

    html += ".status-full { background: #ef5350;

```

```

color: white; }"; // Red

html += ".status-half { background: #ffca28;
color: black; }"; // Yellow

html += ".status-normal { background: #9ccc65;
color: black; }"; // Green

html += ".lid-status-open { background:
#4caf50; color: white; }"; // Green for open

html += ".lid-status-closed { background:
#f44336; color: white; }"; // Red for closed

html += "table { width: 100%; border-collapse:
collapse; margin-top: 10px; }";

html += "th, td { padding: 10px; border: 1px
solid #ddd; text-align: left; }";

html += "th { background: #00796b; color:
white; }";

html += ".table-container { max-height: 200px;
overflow-y: auto; }";

html += "button { padding: 8px 12px; margin:
5px 5px 5px 0; border: none; border-radius: 4px;
cursor: pointer; }";

html += ".btn-open { background-color:
#4CAF50; color: white; }"; // Green button

html += ".btn-close { background-color:
#f44336; color: white; }"; // Red button

html += "</style></head>";

html += "<body>";

html += "<h1>Smart Dustbin Dashboard</h1>";

html += "<div class='container'>";

// Wet Waste Bin

html += "<div class='card wet'>";

html += "<h2>Wet Waste</h2>";

html += "<p>Level: <span
id='wetLevel'>0%</span>";

html += "<span id='wetStatus' class='status
status-normal'>Normal</span></p>";

html += "<div class='progress'><div
id='wetProgress' class='progress-bar'
style='width:0%'></div></div>";

```

```

html += "<p>Lid: <span id='wetLidStatus'
class='lid-status
lid-status-closed'>Closed</span>"; // Lid status

html += "<button class='btn-open'
onclick='controlLid(\"wet\", \"open\")'>Open Wet
Lid</button>"; // Wet Open

html += "<button class='btn-close'
onclick='controlLid(\"wet\", \"close\")'>Close
Wet Lid</button>"; // Wet Close

html += "</p>";

html += "</div>";

// Dry Waste Bin

html += "<div class='card dry'>";

html += "<h2>Dry Waste</h2>";

html += "<p>Level: <span
id='dryLevel'>0%</span>";

html += "<span id='dryStatus' class='status
status-normal'>Normal</span></p>";

html += "<div class='progress'><div
id='dryProgress' class='progress-bar'
style='width:0%'></div></div>";

html += "<p>Lid: <span id='dryLidStatus'
class='lid-status
lid-status-closed'>Closed</span>"; // Lid status

html += "<button class='btn-open'
onclick='controlLid(\"dry\", \"open\")'>Open Dry
Lid</button>"; // Dry Open

html += "<button class='btn-close'
onclick='controlLid(\"dry\", \"close\")'>Close Dry
Lid</button>"; // Dry Close

html += "</p>";

html += "</div>";

// Waste Type and History

html += "<div class='card'>";

html += "<h2>Waste Detection &
Sorting</h2>";

html += "<p>Last Detected Waste Type (for
sorting): <span
id='wasteType'>None</span></p>";

```

```

html += "<h3>Detection History</h3>";
html += "<div class='table-container'>";
html += "<table>";
html += "<thead><tr><th>Time</th><th>Waste  
Type Detected</th></tr></thead>";
html += "<tbody id='historyTable'></tbody>";
html += "</table>";
html += "</div>";
html += "</div>"; // End container
html += "<script>";
html += "function updateData() {";
html += "  fetch('/data').then(response =>  
response.json()).then(data => {";
  // Levels and Status
  html += "    const wetLevel =  
data.wetLevel.toFixed(1);"; // Use 1 decimal  
place for display
  html += "    const dryLevel =  
data.dryLevel.toFixed(1);";
  html += "    document.getElementById('wetLevel').innerText  
= wetLevel + '%';";
  html += "    document.getElementById('dryLevel').innerText  
= dryLevel + '%';";
  html += "    document.getElementById('wetProgress').style.wi  
dth = Math.min(100, Math.max(0, wetLevel)) +  
'%';"; // Clamp width 0-100
  html += "    document.getElementById('dryProgress').style.wi  
dth = Math.min(100, Math.max(0, dryLevel)) +  
'%';"; // Clamp width 0-100
  html += "    const wetStatusEl =  
document.getElementById('wetStatus');";
  html += "    const dryStatusEl =  
document.getElementById('dryStatus');";
  html += "    wetStatusEl.innerText = wetLevel

```

```

>= 95 ? 'Full' : wetLevel >= 50 ? 'Half' :  
'Normal';"; // Adjusted thresholds
  html += "    dryStatusEl.innerText = dryLevel  
>= 95 ? 'Full' : dryLevel >= 50 ? 'Half' :  
'Normal';";
  html += "    wetStatusEl.className = 'status ' +  
(wetLevel >= 95 ? 'status-full' : wetLevel >= 50 ?  
'status-half' : 'status-normal');";
  html += "    dryStatusEl.className = 'status ' +  
(dryLevel >= 95 ? 'status-full' : dryLevel >= 50 ?  
'status-half' : 'status-normal');";
  // Waste Type
  html += "    document.getElementById('wasteType').innerTex  
t = data.wasteType;";
  // Lid Status (NEW)
  html += "    const wetLidStatusEl =  
document.getElementById('wetLidStatus');";
  html += "    const dryLidStatusEl =  
document.getElementById('dryLidStatus');";
  html += "    wetLidStatusEl.innerText =  
data.wetLidOpen ? 'Open' : 'Closed';";
  html += "    dryLidStatusEl.innerText =  
data.dryLidOpen ? 'Open' : 'Closed';";
  html += "    wetLidStatusEl.className = 'lid-  
status ' + (data.wetLidOpen ? 'lid-status-open' :  
'lid-status-closed');";
  html += "    dryLidStatusEl.className = 'lid-  
status ' + (data.dryLidOpen ? 'lid-status-open' :  
'lid-status-closed');";
  html += "  }).catch(err =>  
console.error('Fetch /data error:', err ?  
err.message : 'Unknown error'))"; // Improved  
error logging
  // History
  html += "  fetch('/history').then(response =>  
response.json()).then(data => {";
  html += "    const tableBody =  
document.getElementById('historyTable');";
  html += "    tableBody.innerHTML = '';"; //  
Clear previous entries

```

```

    html += "    data.slice().reverse().forEach(entry
=> {"; // Show newest first

    html += "        const row =
document.createElement('tr');"

    html += "        row.innerHTML = '<td>' +
entry.timestamp + '</td><td>' + entry.type +
'</td>';";

    html += "        tableBody.appendChild(row);";

    html += "    });";

    html += "    }).catch(err =>
console.error('Fetch /history error:', err ?
err.message : 'Unknown error'))";

    html += "    }";

    // NEW: Function to control lids via fetch API

    html += "function controlLid(type, action) {";

    html += "    console.log('Sending command: /lid?
type=' + type + '&action=' + action);";

    html += "    fetch('/lid?type=' + type + '&action='
+ action)";

    html += "        .then(response => {";

    html += "        if (!response.ok) { throw new
Error('Network response was not ok: ' +
response.statusText); }";

    html += "        return response.text();";

    html += "    }";

    html += "        .then(text => {";

    html += "        console.log('Lid control response:',
text);";

    // Optionally, update UI immediately or wait for
the next /data refresh

    // updateData(); // Call immediately to refresh
status - uncomment if desired

    html += "    });";

    html += "        .catch(error =>
{ console.error('Error controlling lid:', error); });";

    html += "    }";

    // Update data every 2 seconds and on load

    html += "setInterval(updateData, 2000);";

```

```

    html += "window.onload = updateData;";
    html += "</script></body></html>";
    return html;
}

// --- Web Server Handlers ---

void handleRoot() {
    Serial.println("Received request for /");
    server.sendHeader("Access-Control-Allow-
Origin", "*");
    server.send(200, "text/html", getHTML());
}

void handleData() {
    //Serial.println("Received request for /data"); //
Reduce serial clutter

    String json = "{";

    json += "\"wetLevel\":" + String(wetLevel, 2) +
",";

    json += "\"dryLevel\":" + String(dryLevel, 2) +
",";

    json += "\"wasteType\":" + lastWasteType +
",";

    json += "\"wetLidOpen\":" +
String(isWetLidOpen ? "true" : "false") + ","; //
NEW

    json += "\"dryLidOpen\":" +
String(isDryLidOpen ? "true" : "false"); // NEW

    json += "}";

    //Serial.println("Sending /data response: " +
json); // Reduce serial clutter

    server.sendHeader("Access-Control-Allow-
Origin", "*");
    server.send(200, "application/json", json);
}

```



```
void handleStatus() { // Kept for compatibility,
mirrors part of /data
```

```
    Serial.println("Received request for /status");
```

```
    String json = "{";
```

```
    json += "\"wasteType\": \"" + lastWasteType +
    "\"";
```

```
    json += "}";
```

```
    Serial.println("Sending /status response: " +
    json);
```

```
    server.sendHeader("Access-Control-Allow-
    Origin", "*");
```

```
    server.send(200, "application/json", json);
```

```
}
```

```
void handleHistory() {
```

```
    //Serial.println("Received request for
    /history"); // Reduce serial clutter
```

```
    String json = "[";
```

```
    // Limit to sending maybe last 10 or 20 for
    performance if log is huge?
```

```
    int startIdx = (detectionCount > 50) ? 0 : 0; //
    Start from beginning for now
```

```
    // int startIdx = max(0, detectionCount - 20); //
    Example: Send last 20
```

```
    for (int i = startIdx; i < detectionCount; i++) {
```

```
        // Check if index is valid within the circular
        buffer logic (if implemented that way)
```

```
        // Current code uses shift, so index i is always
        valid up to detectionCount (max 50)
```

```
        json += "{\"type\": \"" + detectionLog[i].type +
        "\", \"timestamp\": \"" + detectionLog[i].timestamp
        + "\"}";
```

```
        if (i < detectionCount - 1) json += ",";
```

```
    }
```

```
    json += "]";
```

```
    //Serial.println("Sending /history response: " +
    json); // Reduce serial clutter
```

```
    server.sendHeader("Access-Control-Allow-
    Origin", "*");
```

```
    server.send(200, "application/json", json);
```

```
}
```

```
void handlePlainData() { // Kept for compatibility
(e.g., simple app inventor integration)
```

```
    Serial.println("Received request for /plaintext");
```

```
    String plainData = String(wetLevel, 2) + "," +
    String(dryLevel, 2) + "," + lastWasteType;
```

```
    Serial.println("Sending /plaintext response: " +
    plainData);
```

```
    server.sendHeader("Access-Control-Allow-
    Origin", "*");
```

```
    server.send(200, "text/plain", plainData);
```

```
}
```

```
// NEW: Handle requests for manual lid control
```

```
void handleLidControl() {
```

```
    String type = server.arg("type"); // Get 'type'
    parameter (wet/dry)
```

```
    String action = server.arg("action"); // Get
    'action' parameter (open/close)
```

```
    Serial.print("Received /lid request: type=");
```

```
    Serial.print(type);
```

```
    Serial.print(", action=");
```

```
    Serial.println(action);
```

```
    bool success = false;
```

```
    if (type == "wet") {
```

```
        if (action == "open") {
```

```
            wetLidServo.write(LID_OPEN_ANGLE);
```

```
            isWetLidOpen = true;
```

```
            Serial.println("Opening Wet Lid");
```

```
            success = true;
```

```

    } else if (action == "close") {
        wetLidServo.write(LID_CLOSED_ANGLE);
        isWetLidOpen = false;
        Serial.println("Closing Wet Lid");
        success = true;
    }
    } else if (type == "dry") {
        if (action == "open") {
            dryLidServo.write(LID_OPEN_ANGLE);
            isDryLidOpen = true;
            Serial.println("Opening Dry Lid");
            success = true;
        } else if (action == "close") {
            dryLidServo.write(LID_CLOSED_ANGLE);
            isDryLidOpen = false;
            Serial.println("Closing Dry Lid");
            success = true;
        }
    }

    server.sendHeader("Access-Control-Allow-Origin", "*"); // Add CORS header

    if (success) {
        server.send(200, "text/plain", "OK: " + type +
        " lid " + action);
    } else {
        server.send(400, "text/plain", "Error: Invalid
        type or action");
    }
}

// --- Setup ---

void setup() {
    // Initialize serial
    Serial.begin(115200);

    while (!Serial) { delay(10); } // Wait for Serial

```

```

if needed
    Serial.println("\n\n--- Smart Dustbin Initializing
    ---");

    // Initialize pins
    pinMode(WET_TRIG, OUTPUT);
    pinMode(WET_ECHO, INPUT);
    pinMode(DRY_TRIG, OUTPUT);
    pinMode(DRY_ECHO, INPUT);
    pinMode(OBJ_TRIG, OUTPUT);
    pinMode(OBJ_ECHO, INPUT);
    pinMode(MOISTURE_PIN, INPUT); // Default
    is INPUT_PULLUP for analog? Check ESP32
    docs. Usually just INPUT is fine.

    pinMode(GREEN_LED, OUTPUT);
    pinMode(RED_LED, OUTPUT);
    pinMode(BUZZER, OUTPUT);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);

    Serial.println("Basic I/O Pins Initialized.");

    // Initialize sorting servo
    sortingServo.attach(SORTING_SERVO_PIN,
    500, 2500); // Min pulse 500µs, max 2500µs

    Serial.println("Moving sorting servo to 90
    degrees (Neutral)");
    sortingServo.write(90); // Neutral position
    delay(500); // Wait for servo

    Serial.println("Sorting Servo Initialized.");

    // Initialize NEW Lid Servos
    wetLidServo.attach(WET_LID_SERVO_PIN,
    500, 2500);

    dryLidServo.attach(DRY_LID_SERVO_PIN,

```

```

500, 2500);

Serial.println("Moving lid servos to closed
position (" + String(LID_CLOSED_ANGLE) + "
degrees)");

wetLidServo.write(LID_CLOSED_ANGLE);
dryLidServo.write(LID_CLOSED_ANGLE);

isWetLidOpen = false; // Set initial state
tracking

isDryLidOpen = false;
delay(1000); // Wait for servos
Serial.println("Lid Servos Initialized.");
// Connect to WiFi
WiFi.mode(WIFI_STA); // Explicitly set STA
mode
WiFi.begin(ssid, password);
Serial.print("Connecting to WiFi [");
Serial.print(ssid);
Serial.print("]");
int wifi_tries = 0;
while (WiFi.status() != WL_CONNECTED &&
wifi_tries < 30) { // Timeout after ~15 seconds
    delay(500);
    Serial.print(".");
    wifi_tries++;
}

if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nConnected to WiFi!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
} else {
    Serial.println("\nFailed to connect to WiFi.
Check credentials or network. Running without
network features.");

    // Decide how to proceed - maybe just run
    sensor logic without web server?

    // For now, we proceed and server setup will
    likely fail or be inaccessible.
}

// Setup web server routes only if WiFi
connected
if (WiFi.status() == WL_CONNECTED) {
    server.on("/", HTTP_GET, handleRoot); //
Specify GET method
    server.on("/data", HTTP_GET, handleData);
    server.on("/status", HTTP_GET,
handleStatus);
    server.on("/history", HTTP_GET,
handleHistory);
    server.on("/plaindata", HTTP_GET,
handlePlainData);
    server.on("/lid", HTTP_GET,
handleLidControl); // NEW route for lid control

    server.begin();

    Serial.println("Web server started. Open
browser to http://" + WiFi.localIP().toString());
} else {
    Serial.println("Web server not started due to
WiFi connection failure.");
}

Serial.println("--- Initialization Complete ---");
}

// --- Main Loop ---
void loop() {
    if (WiFi.status() == WL_CONNECTED) {
        server.handleClient(); // Handle HTTP
requests frequently if connected
    }

    unsigned long currentMillis = millis();

```

```

// Read waste levels periodically

static unsigned long lastSensorRead = 0;

if (currentMillis - lastSensorRead >=
SENSOR_READ_DELAY) {

    // --- Wet Level Reading ---

    float wetDistance = getDistance(WET_TRIG,
WET_ECHO);

    // Add extra check for highly unlikely readings
(e.g., < 1cm when not full)

    if (wetDistance >= 0) { // Only update if
reading is valid

        wetLevel = calculateLevel(wetDistance);

        // Serial print only if value changes
significantly? Or less often?

        // Serial.print("Wet Dist: ");
Serial.print(wetDistance, 1); Serial.print("cm,
Lvl: "); Serial.println(wetLevel, 1);

    } else {

        // Serial.println("Wet sensor reading error");

    }

    // --- Dry Level Reading ---

    float dryDistance = getDistance(DRY_TRIG,
DRY_ECHO);

    if (dryDistance >= 0) { // Only update if
reading is valid

        dryLevel = calculateLevel(dryDistance);

        // Serial.print("Dry Dist: ");
Serial.print(dryDistance, 1); Serial.print("cm, Lvl:
"); Serial.println(dryLevel, 1);

    } else {

        // Serial.println("Dry sensor reading error");

    }

    // Minimal serial output for levels to reduce
clutter

    static unsigned long lastLevelPrint = 0;

    if (currentMillis - lastLevelPrint > 2000) { //

```

```

Print levels every 2 seconds

        Serial.printf("Levels - Wet: %.1f%%, Dry:
%.1f%%\n", wetLevel, dryLevel);

        lastLevelPrint = currentMillis;

    }

    lastSensorRead = currentMillis;

}

// Handle sorting servo hold non-blocking

if (sortingServoHolding && (currentMillis -
sortingServoHoldStart >=
SORTING_SERVO_HOLD_TIME)) {

    Serial.println("Returning sorting servo to 90
degrees");

    sortingServo.write(90); // Return to neutral

    digitalWrite(REDA_LED, LOW); // Turn off
indicator LEDs

    digitalWrite(GREEN_LED, LOW);

    sortingServoHolding = false;

    lastDetectionTime = currentMillis; // Reset
delay timer after servo returns

}

// Check for object detection for sorting (only if
sorting servo is not currently holding)

if (!sortingServoHolding && (currentMillis -
lastDetectionTime > DETECTION_DELAY)) {

    float objDistance = getDistance(OBJ_TRIG,
OBJ_ECHO);

    // Serial.print("Object Distance: ");
Serial.print(objDistance); Serial.println(" cm"); //
Reduce clutter

    if (objDistance >= 0 && objDistance <
OBJ_THRESHOLD) { // Valid reading and
object detected

        // Object detected, check moisture ONLY IF
LIDS ARE CLOSED (prevent false reads during
manual open)

        if (!isWetLidOpen && !isDryLidOpen) {

```

```

    int sensor_analog =
    analogRead(MOISTURE_PIN);

    // Convert analog reading to percentage (0-
    4095 range for ESP32 ADC)

    // Lower reading often means MORE
    moisture (less resistance)

    // Adjust the formula based on your sensor's
    behavior. Assuming 4095 is dry, 0 is very wet.

    float moisture_percentage = (100.0 -
    (sensor_analog / 4095.0) * 100.0);

    // Clamp percentage 0-100

    moisture_percentage = max(0.0f,
    min(100.0f, moisture_percentage));

    Serial.print("Object detected! Dist: ");
    Serial.print(objDistance, 1);

    Serial.print(" cm. Moisture Reading (0-
    4095): "); Serial.print(sensor_analog);

    Serial.print(", Calculated Percentage: ");
    Serial.print(moisture_percentage, 1);
    Serial.println("%");

    prevWasteType = lastWasteType; // Store
    previous before updating

    if (moisture_percentage >
    MOISTURE_THRESHOLD) {

        // Wet waste

        lastWasteType = "Wet";

        Serial.println("-> Wet waste detected,
        moving sorting servo to 145 degrees");

        digitalWrite(RED_LED, HIGH); // Red
        LED for wet (or choose another color)

        digitalWrite(GREEN_LED, LOW);

        sortingServo.write(145); // Rotate to wet
        waste bin direction

        digitalWrite(BUZZER, HIGH);
        delay(200); digitalWrite(BUZZER, LOW); //
        Short beep

    } else {

        // Dry waste

```

```

        lastWasteType = "Dry";

        Serial.println("-> Dry waste detected,
        moving sorting servo to 30 degrees");

        digitalWrite(GREEN_LED, HIGH); //
        Green LED for dry

        digitalWrite(RED_LED, LOW);

        sortingServo.write(30); // Rotate to dry
        waste bin direction

        digitalWrite(BUZZER, HIGH);
        delay(200); digitalWrite(BUZZER, LOW); //
        Short beep

    }

    // Log detection
    if (detectionCount < 50) {

        detectionLog[detectionCount].type =
        lastWasteType;

        detectionLog[detectionCount].timestamp
        = getTimestamp();

        detectionCount++;

    } else {

        // Shift log to keep newest 50
        for (int i = 0; i < 49; i++) {
            detectionLog[i] = detectionLog[i + 1];
        }

        detectionLog[49].type = lastWasteType;
        detectionLog[49].timestamp =
        getTimestamp();

    }

    sortingServoHoldStart = currentMillis; //
    Start timer to hold servo position

    sortingServoHolding = true;

} else {

    Serial.println("Object detected, but lids are
    open. Skipping moisture check.");

    lastDetectionTime = currentMillis; // Still
    reset delay timer to avoid immediate re-check

}

```

```

}

// No else needed here, if no object is detected,
loop continues and checks again after delay

}

```

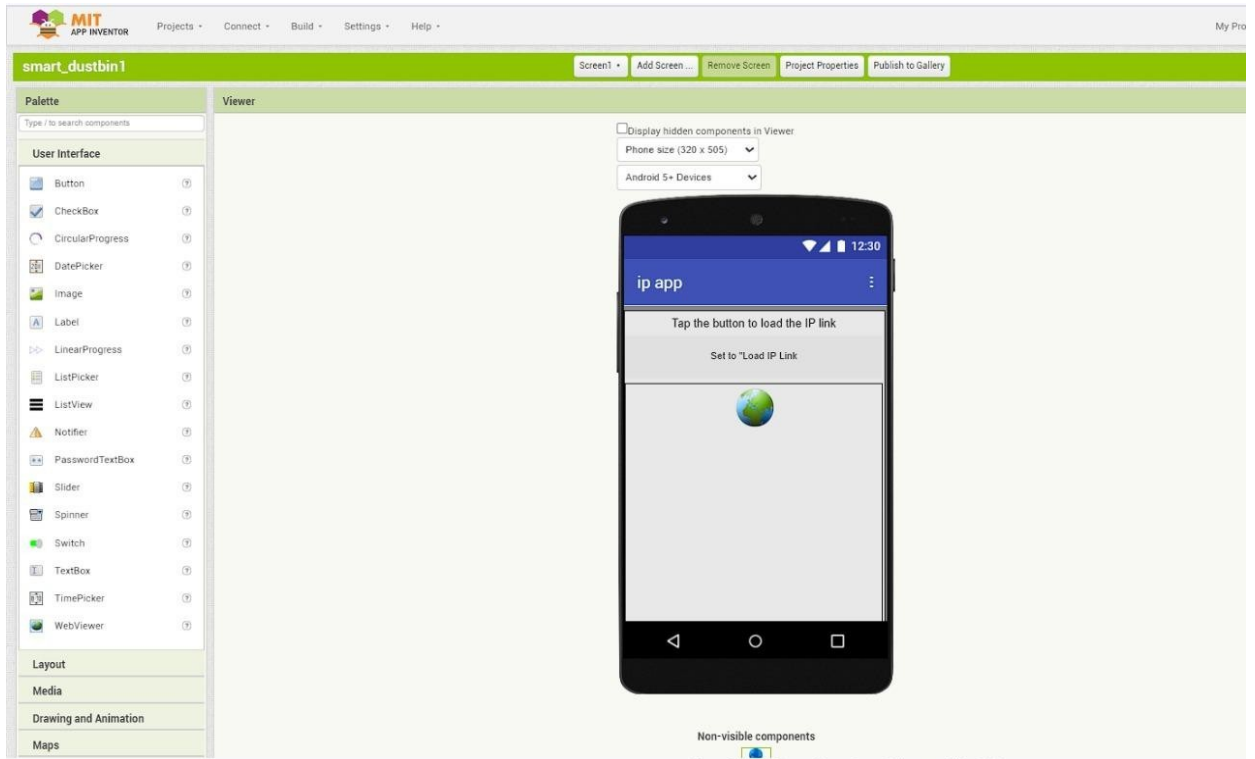
```

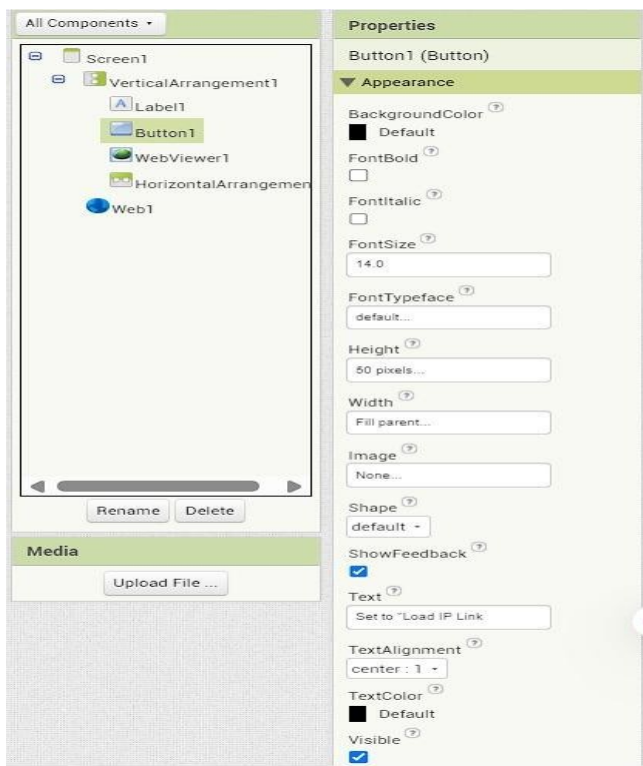
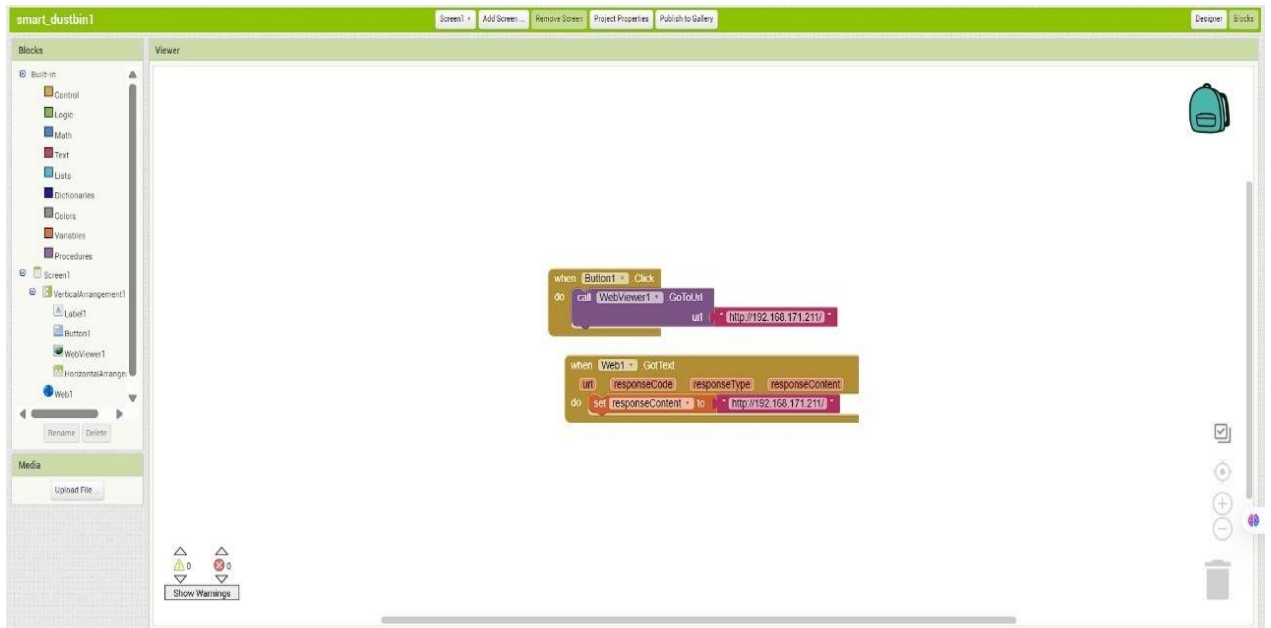
// Add a small delay to prevent loop from
running too fast, allowing ESP32 tasks to run
delay(10);

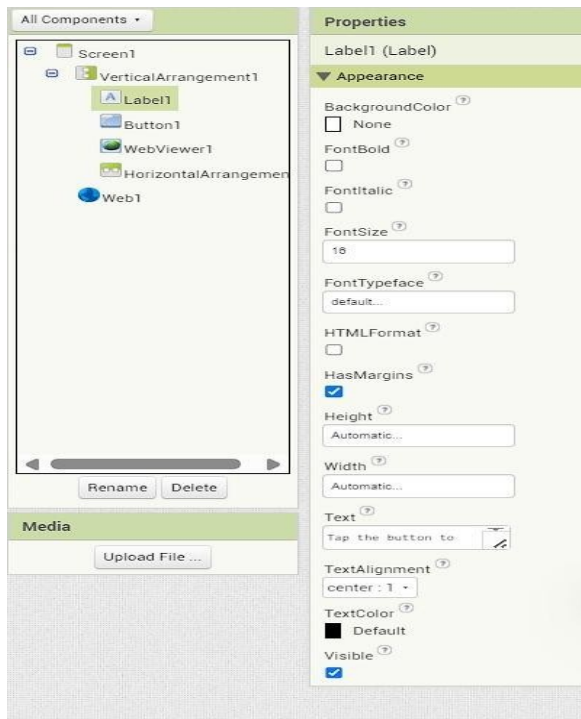
}

```

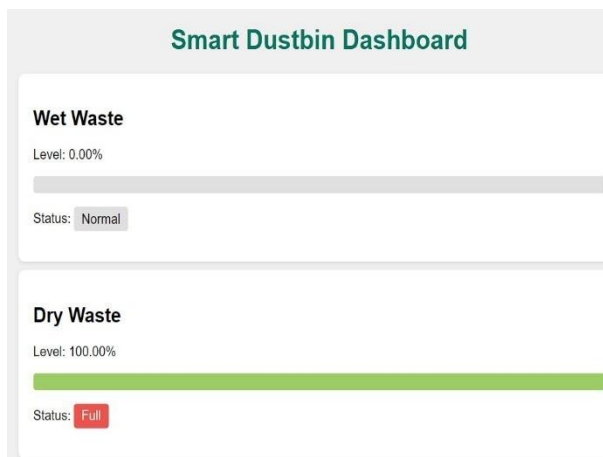
MIT APP DESIGNER:



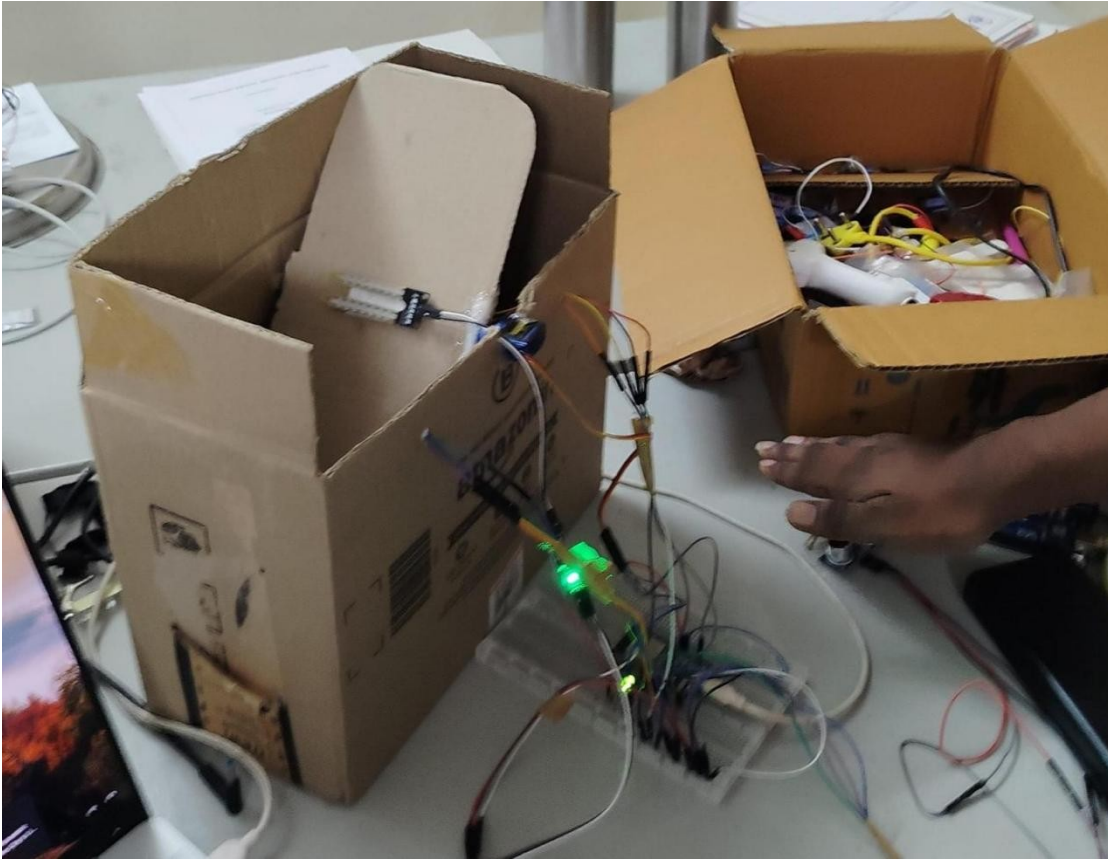




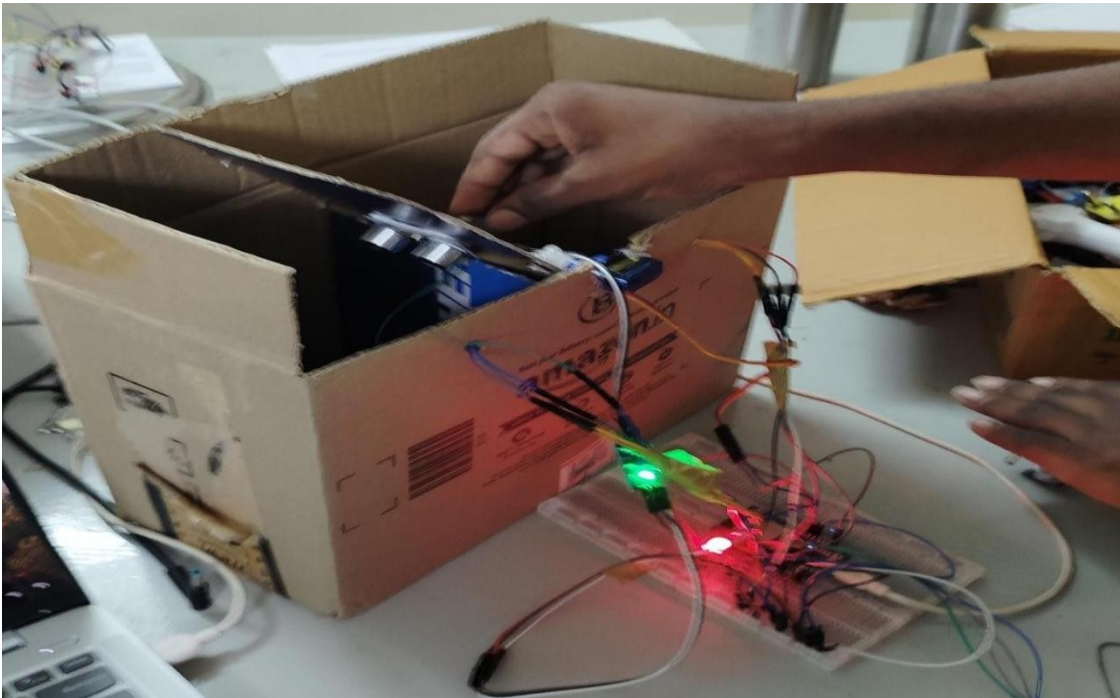
OBSERVATIONS:



DRY WASTE DETECTION :



WET WASTE DETECTION:



RESULT ANALYSIS:

The IoT-based Smart Dustbin was successfully implemented using an ESP32 microcontroller integrated with ultrasonic sensors, a moisture sensor, servo motor, LEDs, a buzzer, and a web dashboard built with Tailwind CSS. The system was able to detect incoming waste, classify it as wet or dry based on moisture levels, and direct it to the corresponding bin using a servo motor. Real-time waste levels were measured and visually displayed on a responsive web interface, along with detection history. The web dashboard effectively communicated the system status over Wi-Fi, proving the feasibility and reliability of remote monitoring and automation in smart waste management.

CONCLUSION:

The Smart Dustbin with IoT is a step forward to automate and modernize waste management systems. The Smart Dustbin combines several sensors such as ultrasonic sensors that measure waste and level sensors that monitor waste levels, and a microcontroller, such as Arduino or ESP modules, to communicate the information. It enables the monitoring of waste in real time more efficiently. This smart bin allows the municipal authorities or private organizations to remotely monitor the condition of each bin and enables timely collection efforts and proper maintenance of the waste management system to avoid issues associated with overflowing and unsanitary bins. The integration of cloud applications and mobile applications provide app users or administrators with alerts and easy command over useful data displayed in graphics on dashboards. This intelligent technology can significantly reduce demand on the work force repeating tasks by reducing the excessive fuel use and time by collecting waste following the same route, which will be optimized using waste collection status. the Smart Dustbin is aligned with the goal of smart cities and sustainable living. It will increase sustainability while cleaning up the urban backyard environment around the community, while creating responsible disposal habits in the public domain. The project indicates several gaps in emerging technologies using IoT to solve our problems to resolve in the interest of responsible waste collection.

PROBLEMS FACED DURING THE IMPLEMENTATION (IF ANY):

1. Internet Connectivity
2. Sensor Accuracy and Placement
3. Software Bugs
4. Making app which works in offline

FUTURE SCOPE:

1. Smart Cities integration
2. Public Spaces & High-Traffic Area
3. Commercial and Industrial Applications
4. Advanced Waste Segregation and Recycling
5. Smart Dustbins in Hospitals and Healthcare